

Module 4

Reinforcement Learning (Q-learning)

Resources

Sutton & Barto – Reinforcement Learning: An Introduction

David Silver Lectures (look through at least lecture 1)

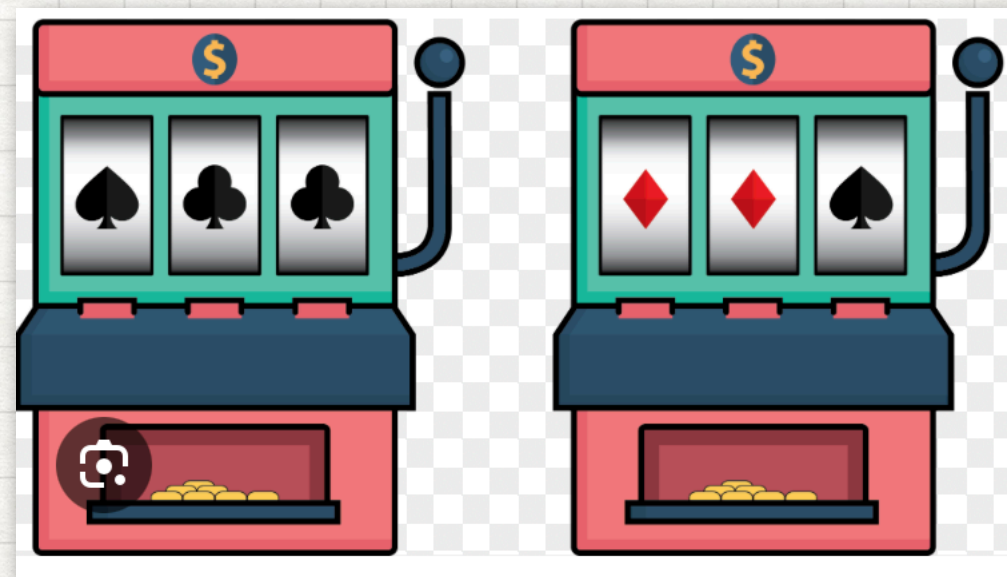
Reinforcement learning models of the basal ganglia and dopamine (e.g., Schultz et al., 1997)

Reinforcement Learning

- Agent exploring the world, learning based on reward
- Behaviour (Policy: $\pi(s)$)
- Storing value of states (v_{π})
- Model of the environment
- Different from supervised learning / unsupervised learning

Reinforcement Learning Example

2-armed Bandit task (slot machine)

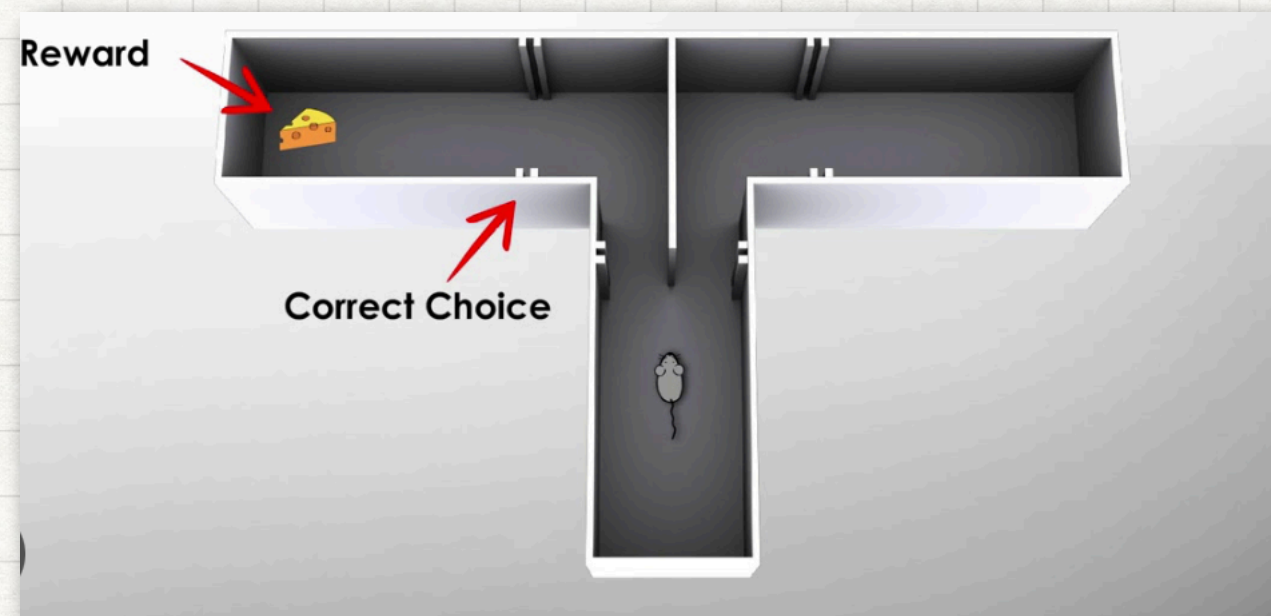


Decision: Which lever to pull if every time you have to pay

Goal: Maximise reward

Reinforcement Learning Example

2-armed Bandit task (T-maze version)

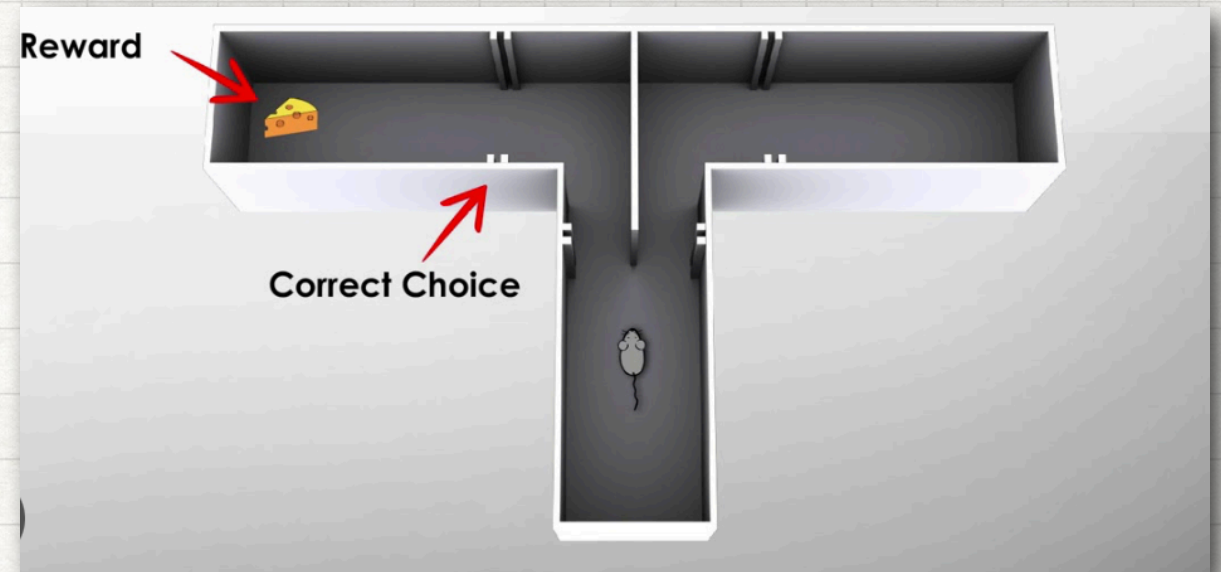


Decision: Which side of the T-maze to choose

Goal: Maximise survival (average reward)

Reinforcement Learning Example

2-armed Bandit task



Rich task, that can be used to examine many aspects of decision-making:

- Learning (learning rates, bias, optimality, uncertainty, number of choices...)
- Exploration vs Exploitation
- Non-stationary world

Reinforcement Learning Example

2-armed Bandit task: Details

Uncertainty in outcomes:

Control the difficulty of the task



p_0



p_1

Nonstationarity:

These probabilities can change with time

Agent estimates the value of each arm: Q-value
(Internal estimate of how good each choice is)

Learning can be fast $\rightarrow$ volatile,
Or stable $\rightarrow$ slow

Learning rate (α)

Action can be best possible (exploit),
or geared at learning (explore)

ϵ -Greedy

Mini-project 4: 2-arm bandit task

Implement Q-learning in a 2-arm bandit task; Plot reward curves

Psychologists and neuroscientists have used this task for decades to study:

- **Reward learning and habit formation**

- how people (or animals) learn from reinforcement signals over time.

- **Exploration vs. exploitation**

- when to *try new* options versus *stick with what works*.

- **Learning under uncertainty**

- how we estimate the value of choices when rewards are probabilistic.

Mini-project 4: 2-arm bandit task

2-armed Bandit task: Outcomes

We would like to observe:

impact of environmental and cognitive parameters on behaviour

Strategy:

Plot actions as a function of time for different parameter values

Plot (moving) average reward as a function of time for different parameters

Prediction:

Initial exploration phase

Acquisition phase

Asymptotic phase

Demo 1: One-arm bandit

Write Python code to implement the environment — that is, a “one-arm bandit” (slot machine)

- Write a class **OneArmBandit** that has an object of class **BanditArm**
- Write a class **BanditArm** that stores the probability of success, p_{success}
- Write a method **pull()** for **BanditArm**: Each time the arm of the bandit is pulled, it should return a reward with probability p .
- Test this bandit, by pulling the arm n times and recording the average reward

Demo 2: Implement Agent (Value Estimate)

Write Python code to implement an agent that learns the value of a bandit arm through experience:

- Write a class **ValueLearner**:
- Attributes:
 - $\alpha \rightarrow$ learning rate
 - q -value $\rightarrow$ the agent's expected value
- Method:
 - `update(reward)` $\rightarrow$ Implements Rescorla-Wagner learning rule to update q -value
- Simulate learning and print the q -values over time

Demo 3: Two-armed bandit

Write Python code to extend the OneArmBandit to a two-arm bandit environment. Also change ValueLearner to update q-value of the arm based on action:

- Write a class **TwoArmedBandit**:
 - Attribute: arms → list of two BanditArm objects
 - Method: sample(action) → Calls the pull() method of arm based on 'action'
- Modify the class **Agent**:
 - Change attribute 'q-value' to array (initialised to 0)
 - Change the 'update' method to update q-value based on action
 - Add an 'act' method that chooses which arm to sample (just choose best)

Demo 4: Improving the act() method

ϵ -Greedy

Write Python code to improve the act() method so that it mostly samples the best action (exploit) but sometimes samples one of the other actions (explore):

- Modify the ValueLearner class to contain:
 - Attribute ϵ
- Modify the act() method to:
 - Choose the action where q_value is max(q_value) with probability $(1-\epsilon)$
 - Choose one of the other actions with probability ϵ
- Vary ϵ and see effect on learning

Reinforcement Learning Example

2-armed Bandit task: Details

Uncertainty in outcomes:

Control the difficulty of the task



p_0



p_1

Nonstationarity:

These probabilities can change with time

Agent estimates the value of each arm: Q-value
(Internal estimate of how good each choice is)

Learning can be fast $\rightarrow$ volatile,
Or stable $\rightarrow$ slow

Learning rate (α)

Action can be best possible (exploit),
or geared at learning (explore)

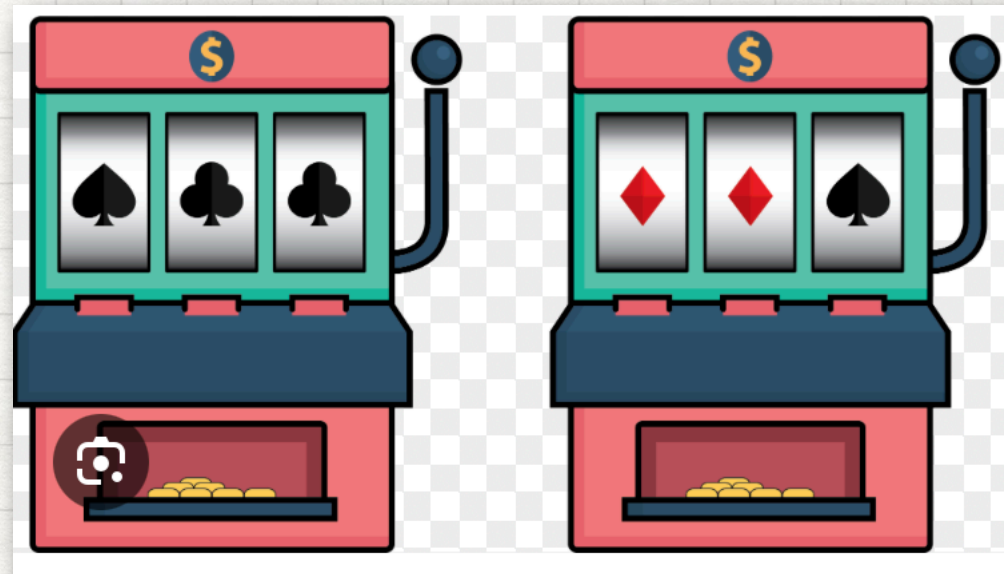
ϵ -Greedy

Reinforcement Learning Example

2-armed Bandit task: Changing world

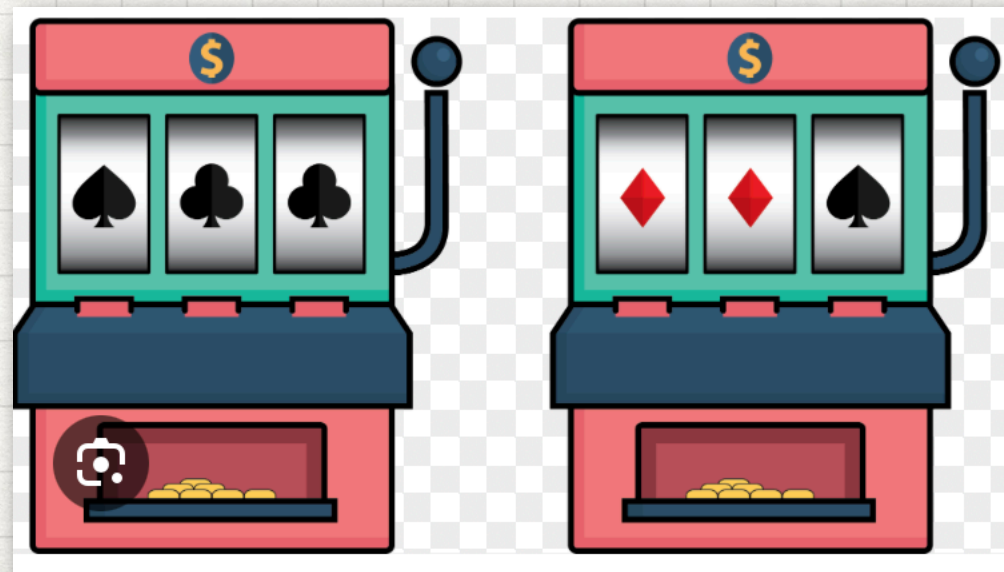


p_{0A}

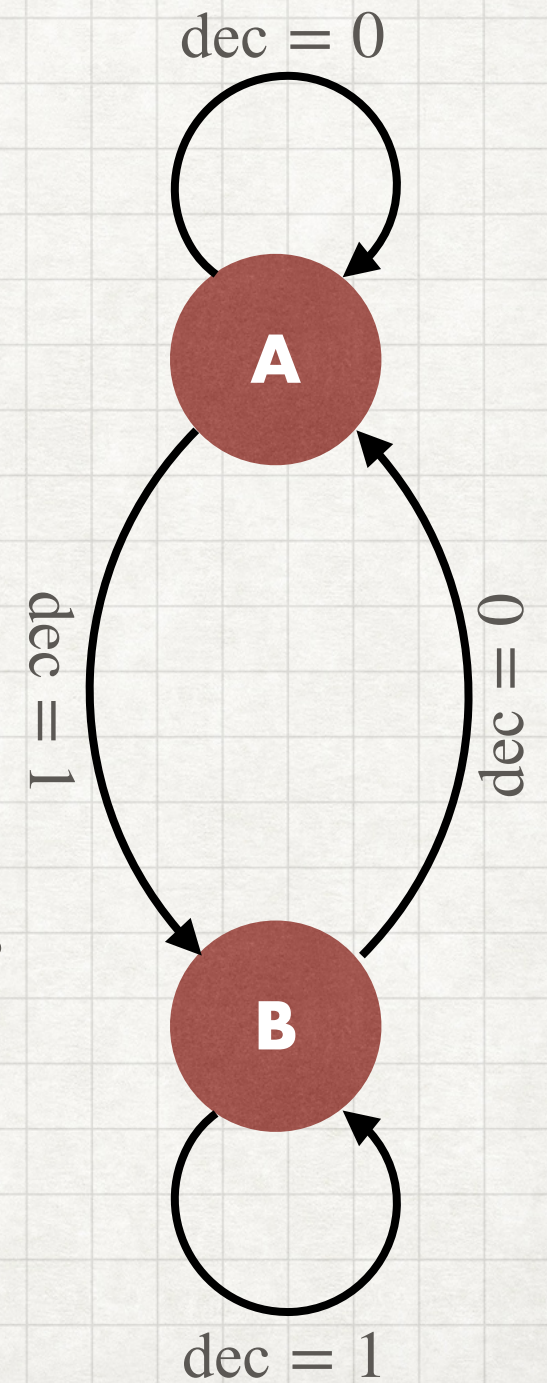


p_{1A}

p_{0B}



p_{1B}

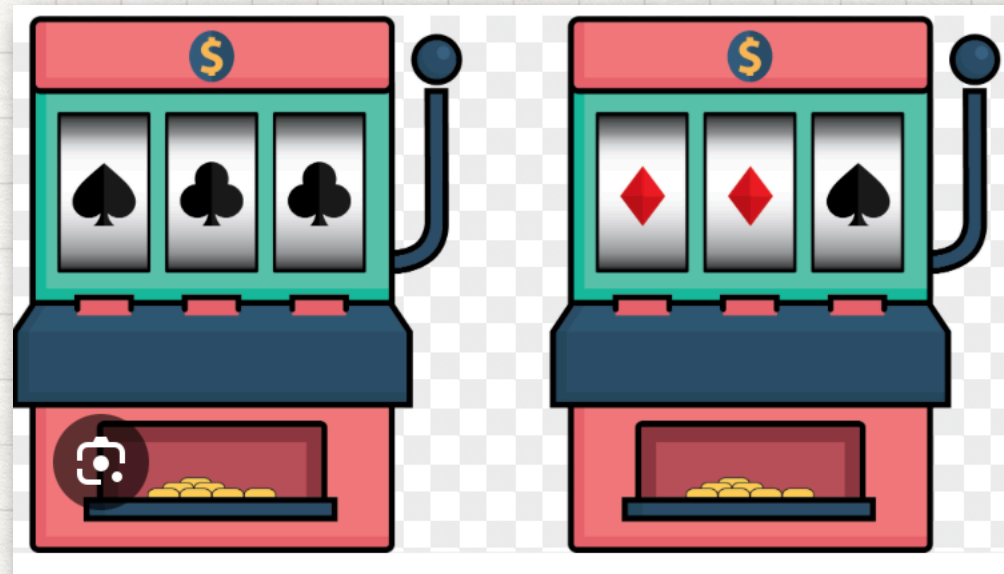


Reinforcement Learning Example

2-armed Bandit task: Changing world



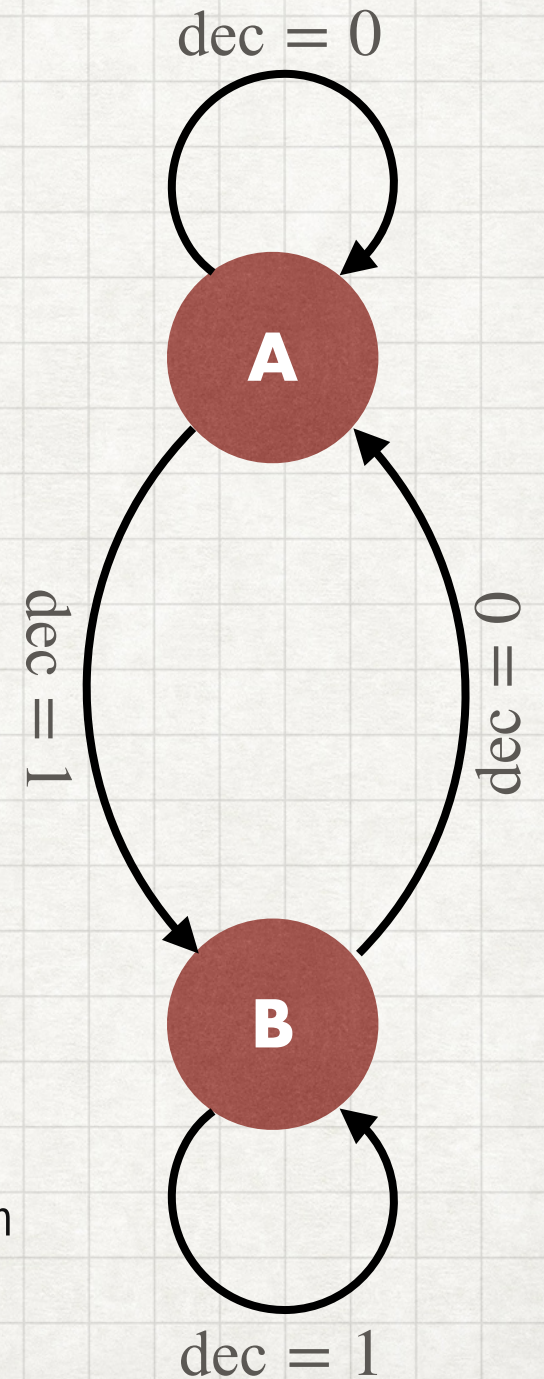
p_{0A}



p_{1A}

Consider an agent in State A...

- Previously the agent just learned based on best action (highest reward)
- But now...
- The agent must also consider it's current state
- If making choosing arm=1 gives it low reward, but takes it to State=B, which has high reward for an action, maybe the agent should choose arm=1



Reinforcement Learning Example

Changing the Learning Rule for a changing world

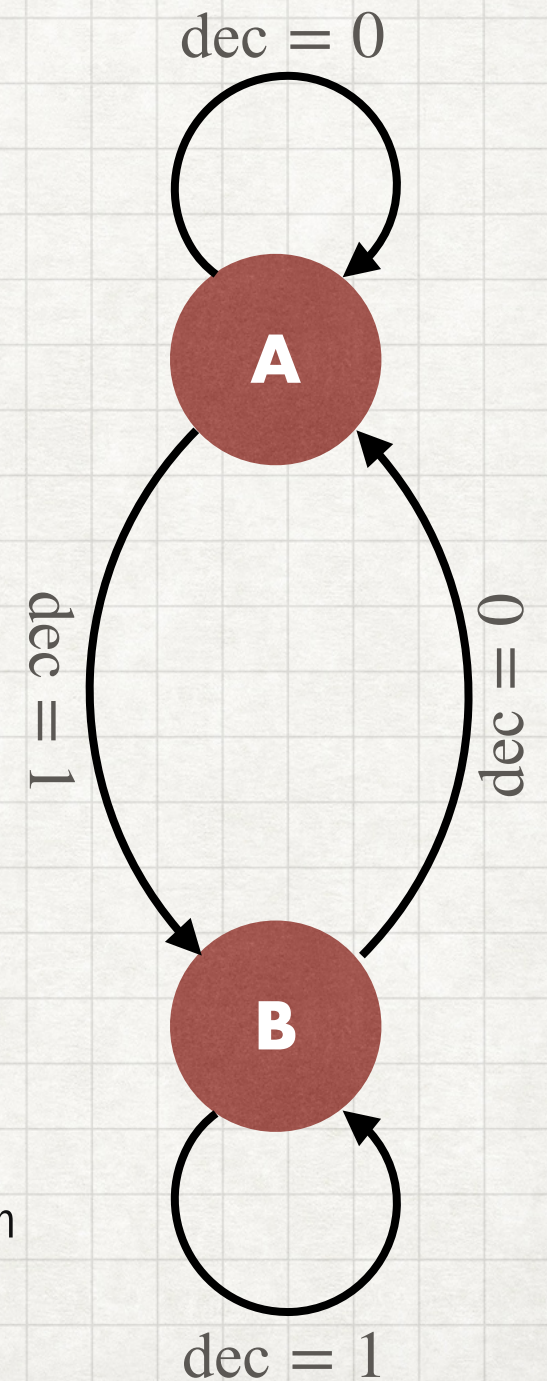
Previous learning rule:

$$Q(a) \leftarrow Q(a) + \alpha[r - Q(a)]$$

What should the new learning rule be?

Consider an agent in State A...

- Previously the agent just learned based on best action (highest reward)
- But now...
- The agent must also consider it's current state
- If making choosing arm=1 gives it low reward, but takes it to State=B, which has high reward for an action, maybe the agent should choose arm=1



Reinforcement Learning Example

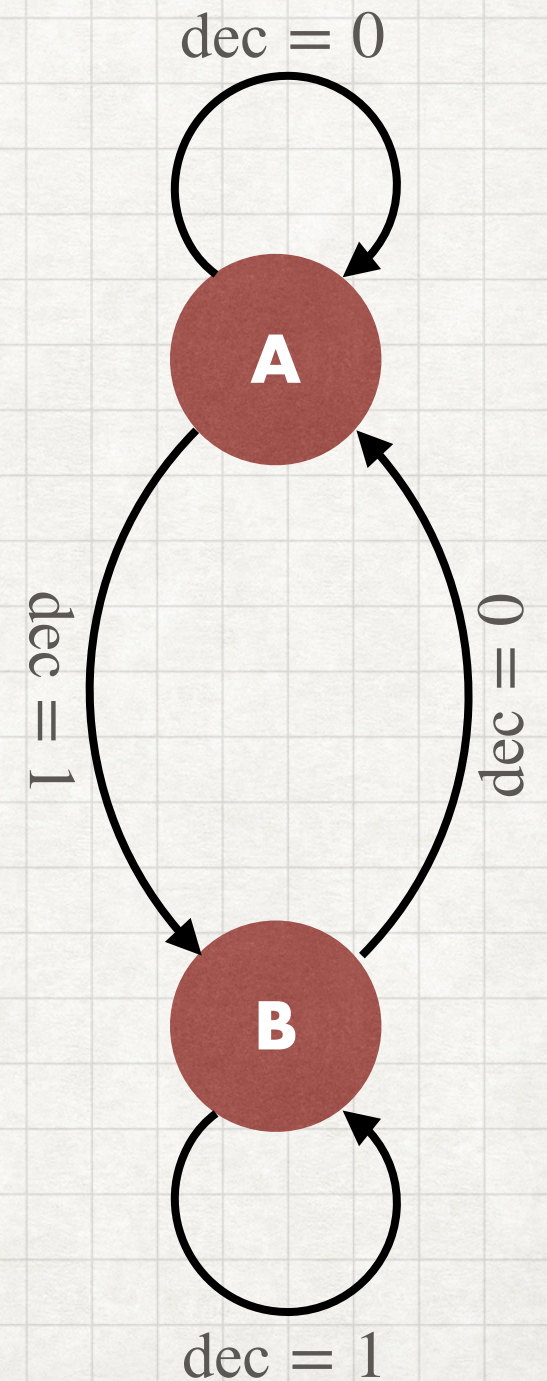
Changing the Learning Rule for a changing world

A new meaning of Q-value

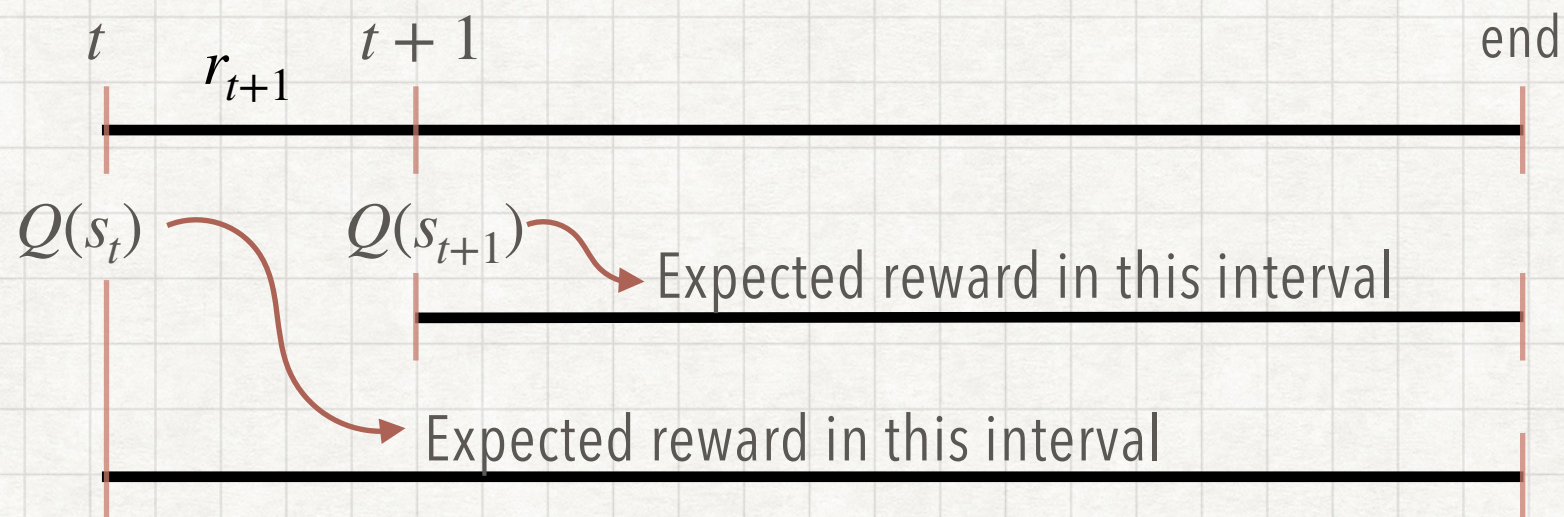
$Q(s_t, a) \leftarrow$ The expected reward from here (t) to the end of time

$Q(s_{t+1}, a) \leftarrow$ The expected reward from ($t + 1$) to the end of time

$r_{t+1} \leftarrow$ The reward that actually came to pass b/w t and ($t + 1$)



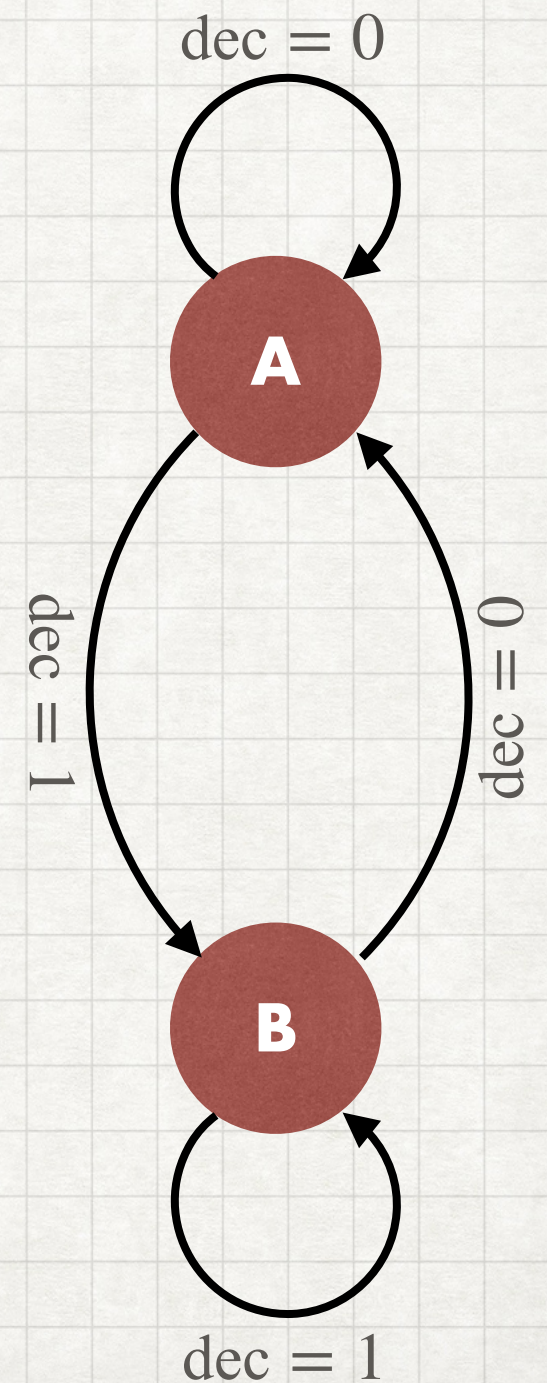
Reinforcement Learning Example



Key Idea: What a difference a time-step makes!

- Update $Q(s_t, a)$ based on the reward that happened between t & $t+1$
- Generate an expected reward based on current value of $Q(s_t, a)$ and $Q(s_{t+1}, a)$, and compare it to r_{t+1}
- Error in prediction: $Q(s_t, a)$ and $Q(s_{t+1}, a)$, and compare it to r_{t+1}
- That is, $\text{Error} = r_{t+1} - [Q(s_t, a) - Q(s_{t+1}, a)]$

Temporal Difference Learning



Reinforcement Learning Example

Temporal Difference Learning

Previous learning rule:

$$Q(a) \leftarrow Q(a) + \alpha[r - Q(a)]$$

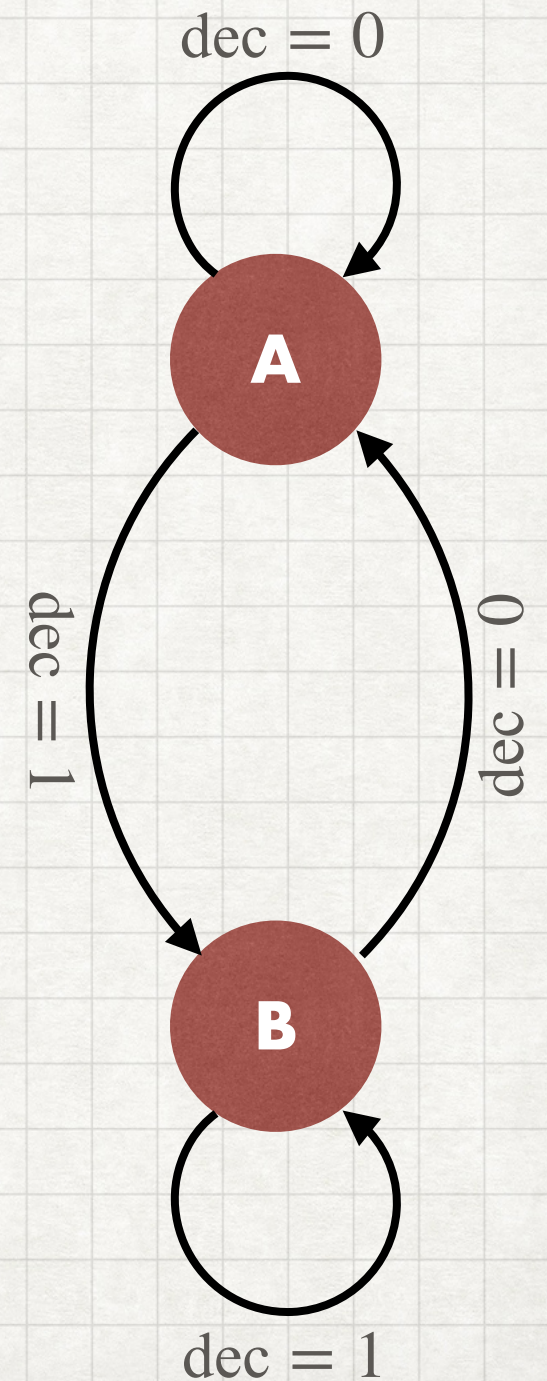
New learning rule:

$$Q(s_t, a) \leftarrow Q(s_t, a) + \alpha[r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a)]$$

discounting factor

TD Error

Assume agent takes best possible action in next time step (**off-policy**)



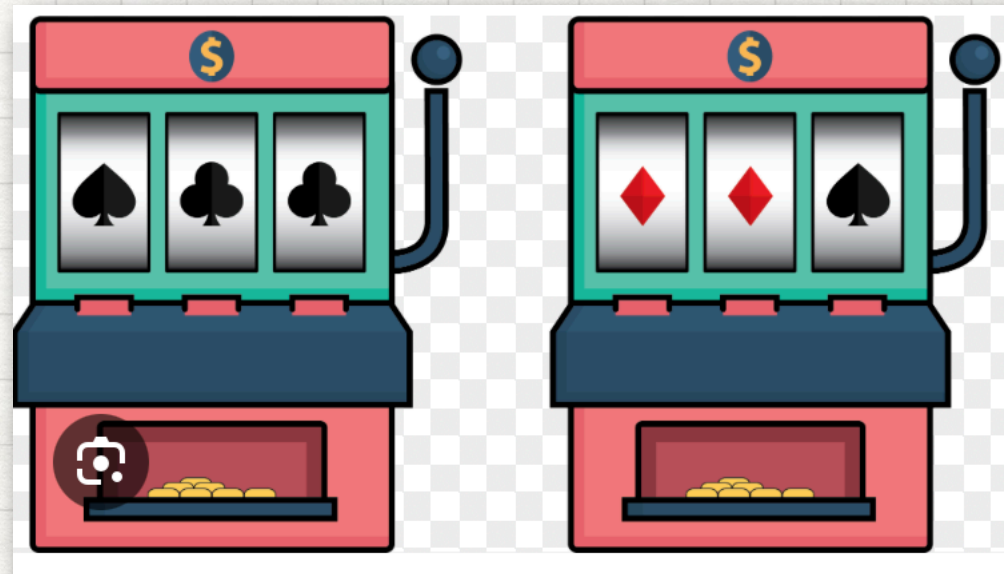
Two take-home assignments

1. The world can change with time

p_0 and p_1 can change with time



p_0



p_1

Find an algorithm to change the world in an interesting manner and plot the Q-value curves and discuss the suitability of Q-learning / TD learning

2. The non-best option can be explored in a more interesting manner

ϵ -Greedy is inefficient



Research better algorithms to do explorations (other than what we discuss today), implement and discuss results.

Exploration algorithm 2

Annealed ϵ -Greedy

Remember this?

$$\frac{e^{x_i}}{\sum e^{x_i}} \longrightarrow \frac{e^{\frac{x_i}{\tau}}}{\sum e^{\frac{x_i}{\tau}}}$$

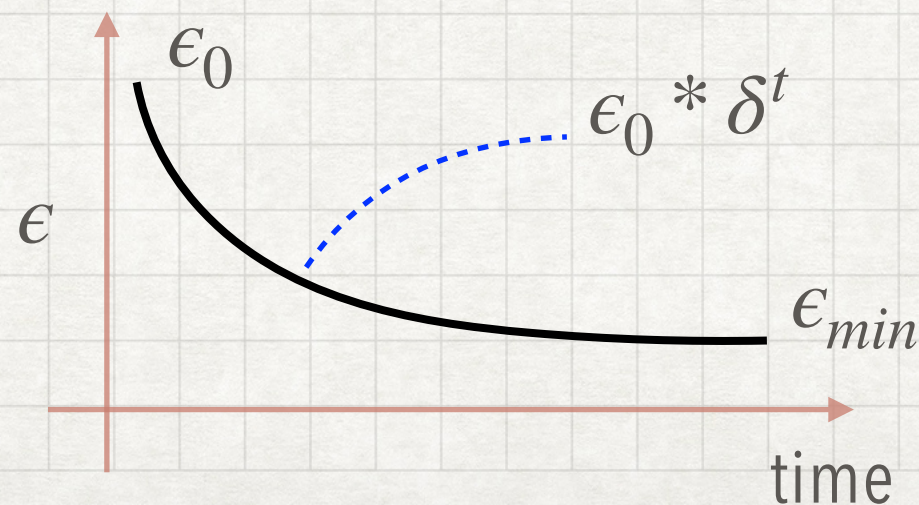
temperature

Current ϵ -Greedy rule:

$$\text{decision} = \begin{cases} \text{best action,} & \text{w.p. } 1 - \epsilon \\ \text{one of other actions,} & \text{w.p. } \epsilon \end{cases}$$

Fixed!

Annealed ϵ -Greedy rule:



Exploration algorithm 3

Softmax

Remember this?

$$\frac{e^{x_i}}{\sum e^{x_i}} \longrightarrow \frac{e^{\frac{x_i}{\tau}}}{\sum e^{\frac{x_i}{\tau}}}$$

temperature

Softmax rule:

$$\text{decision} = \begin{cases} a_1, & \text{w.p. } e^{\frac{Q_1}{\tau}} \\ a_2, & \text{w.p. } e^{\frac{Q_2}{\tau}} \end{cases}$$

Hint:

Sample with custom probabilities:

python

```
import numpy as np
my_array = [10, 20, 30, 40, 50]
probabilities = [0.1, 0.1, 0.1, 0.3, 0.4]
weighted_sample = np.random.choice(my_array, p=probabilities)
# Example output: 50 (higher chance)
```